

MODERN WEB DEVELOPMENT

From HTML to Full-Stack

Course Title: Desktop & Web Programming

Presented By:

Adnan Zahan Romeo (242-35-215)

Faysal Ahmed Shuvo (242-35-576)

Abu Rayhan (242-35-077)

Al-Shahrier Shihab (242-35-732)

Presented To:

Pranto Saha

Designation: Lecturer

Date: 16-08-2026

The Evolution of the Web

From documents you read to applications you use.

1991–99

Static Web

Plain HTML documents, hyperlinks,
simple hand-coded pages.

2006–11

Rich Interactivity

AJAX, jQuery — pages update
without a full reload.

2018–Now

Full-Stack & Beyond

Meta-frameworks, edge rendering,
AI-assisted, type-safe stacks.

2000–05

Dynamic Sites

Server-rendered pages (PHP, ASP),
early CSS layouts, databases.

2012–17

The SPA Era

React, Angular, Vue — apps run
largely in the browser.

Frontend Fundamentals

Every website — no matter how advanced — still rests on three core technologies.



STRUCTURE

HTML

Semantic markup that defines content, meaning, and accessibility.



PRESENTATION

CSS

Layout, typography, responsiveness, animation — Flexbox & Grid.



BEHAVIOR

JavaScript

Interactivity, state, DOM updates, and app logic in the browser.

Modern Frontend Frameworks

Frameworks bring structure, reusability, and state management to complex UIs.

React

Component-based library from Meta. Huge ecosystem, virtual DOM, powers Next.js.

Vue

Approachable, incrementally adoptable framework with a gentle learning curve.

Angular

Full-featured, opinionated framework built for large enterprise applications.

Svelte

Compiles away at build time — no virtual DOM, minimal runtime overhead.

Choosing a framework is a trade-off between ecosystem size, learning curve, and performance needs.

Backend Development & APIs

The server handles logic, data, and delivers responses that power the frontend.



Server Runtimes

Node.js, Python (Django/Flask), Java (Spring), Go, Ruby on Rails power business logic.



REST APIs

Resource-based endpoints over HTTP — the most common way clients talk to servers.



GraphQL

A query language letting clients request exactly the data shape they need.



Auth & Sessions

Tokens (JWT), OAuth, and sessions keep requests secure and identity-aware.

The Data Layer

Where an application's information actually lives.

RELATIONAL

SQL

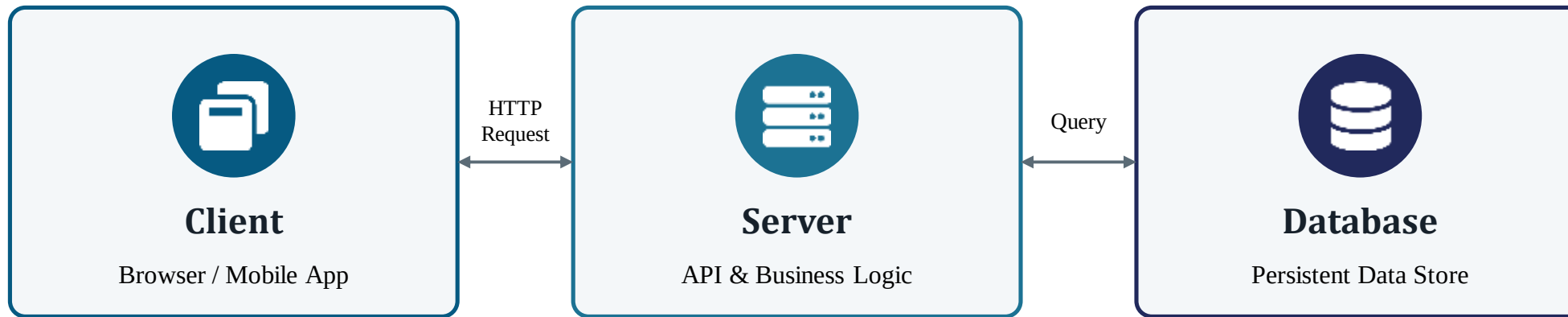
- Structured tables with defined schemas
- Strong consistency (ACID transactions)
- Powerful joins across related data
- Examples: PostgreSQL, MySQL, SQL Server

NON-RELATIONAL

NoSQL

- Flexible, schema-less documents or key-values
- Built to scale horizontally with ease
- Great fit for unstructured or fast-changing data
- Examples: MongoDB, Redis, DynamoDB

How a Full-Stack App Fits Together



Requests flow client → server → database, and responses (JSON, HTML, or files) travel back the same path.

CDN / Edge Cache

Load Balancer

Authentication

Caching Layer

Tooling & Developer Workflow

The tools around the code matter as much as the code itself.



Version Control

Git & GitHub/GitLab track history and enable collaboration.



Type Safety

TypeScript catches errors at compile time, not in production.



Build Tools

Vite, Webpack & esbuild bundle and optimize code for shipping.

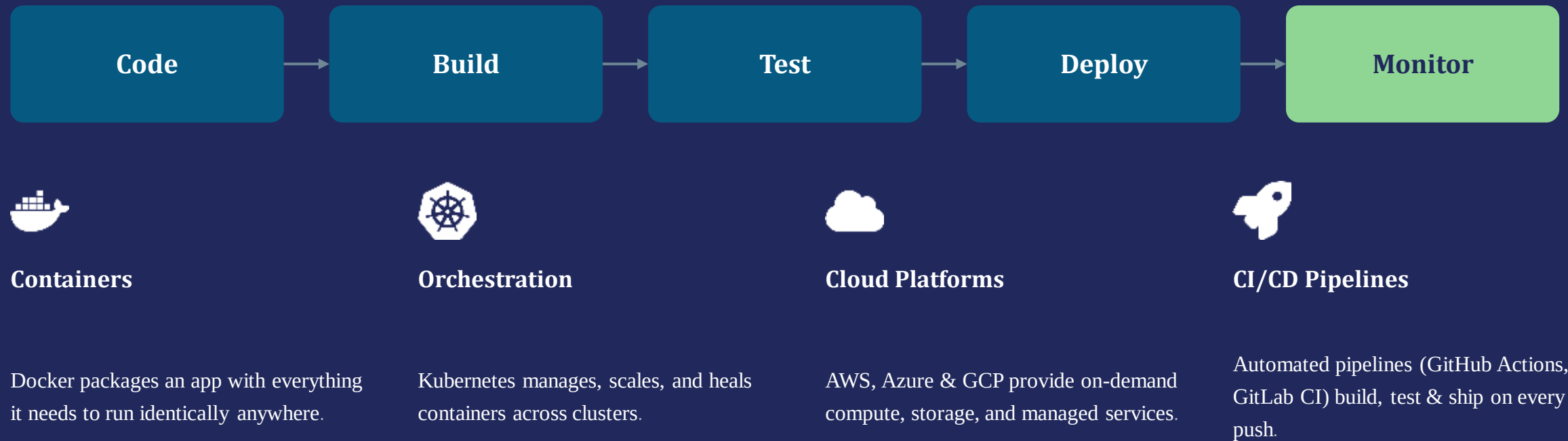


Testing

Unit, integration & E2E tests (Jest, Playwright) catch regressions.

DevOps & Deployment

Getting code from a laptop to production, safely and repeatably.



Performance, Security & What's Next

Non-Negotiables

- Core Web Vitals — measure real-world loading, interactivity & visual stability
- HTTPS everywhere & secure headers (CSP, CORS) by default
- Input validation & sanitization to prevent injection attacks
- Lazy loading, caching & code splitting to keep pages fast
- Regular dependency audits to catch known vulnerabilities

On the Horizon



AI-Assisted Development

Copilots that scaffold, review & debug code alongside developers.



WebAssembly

Near-native performance for demanding tasks, directly in the browser.



Edge Computing

Code running physically closer to users for lower latency.



Serverless

Functions that scale automatically without managing infrastructure.

Key Takeaways



It's Layers, Not Magic

Every complex app is HTML, CSS & JS underneath — frameworks just organize the work.



Full-Stack Means Connected

Frontend, backend & database each do one job well, communicating over APIs.



Ship It Reliably

Testing, CI/CD & monitoring turn code into a dependable product.

Thank You

Questions & discussion welcome.